CAI 6108: Machine Learning Engineering

Project Report: Multi-Label Image Classification via Transfer Learning

Sanjith Devineni
(Point of Contact)
sdevineni@ufl.edu

Sai Neha Ratakonda
ratakondasaineha@ufl.edu

Abhignan Sai Arcot
abhignansaiarcot@ufl.edu

Sahith Reddy Gopidi
sgopidi@ufl.edu

Akshay Jaidi
akshayjaidi@ufl.edu

April 24, 2026

1 Introduction

Multi-label image classification is a fundamental computer vision problem in which a single image may simultaneously belong to multiple semantic categories. Unlike standard single-label classification—where exactly one class is assigned per sample—the multi-label setting requires a model to independently predict the presence or absence of every class in the label set. This substantially increases the difficulty of the problem: the output space grows exponentially with the number of labels, standard accuracy metrics no longer capture partial correctness, and methods based on a single softmax distribution are inapplicable because class memberships are not mutually exclusive. This project addresses a practical multi-label classification task involving twelve everyday object categories found in desk and workspace environments: *pen*, *paper*, *book*, *clock*, *phone*, *laptop*, *chair*, *desk*, *bottle*, *keychain*, *backpack*, and *calculator*.

The dataset consists of 4,543 RGB images, each at a resolution of 128×128 pixels. Instead of a separate annotation file, class memberships are encoded directly in the name of each image's parent directory using an underscore-separated convention. For example, images stored in the folder `pen_paper_book/` carry the labels *pen*, *paper*, and *book*. The images depict real-world desk and workspace scenes in which multiple objects co-occur naturally, providing a rich multi-label structure: images carry between one and seven simultaneous labels, with an average of 1.76 labels per image. Label frequencies are moderately imbalanced—*pen* appears in 765 images while *clock* appears in only 503—introducing a slight challenge of maintaining strong performance across both frequent and rare classes.

We design and implement a complete machine learning pipeline to address this. The pipeline encompasses automated data loading via filesystem inspection with Python's `pathlib` library, multi-hot label vector construction, an 80%/10%/10% train/validation/test split, a logistic regression baseline using flattened pixel features, and a fine-tuned EfficientNet-B0 convolutional neural network [4] leveraging transfer learning from ImageNet. The pipeline is evaluated using four complementary metrics—Exact Match Accuracy, Hamming Loss, Macro F1, and Micro F1—that together assess performance from both strict and lenient perspectives. The fine-tuned EfficientNet-B0 achieves a test-set Macro F1 of 0.8127, compared to 0.1614 for the logistic regression baseline.

2 Approach

Pipeline Overview

The complete machine learning pipeline is implemented in a single Jupyter notebook (`main.ipynb`) using PyTorch [2] and scikit-learn [3]. It proceeds in the following stages: (1) filesystem scan and label parsing; (2) exploratory data analysis; (3) image preprocessing and data augmentation; (4) train/validation/test splitting and DataLoader construction; (5) baseline model training and evaluation; (6) EfficientNet-B0 training with periodic checkpointing; and (7) final evaluation on the held-out test set.

Data Loading and Preprocessing

Data loading requires no external annotation file. Python's `pathlib.Path` library scans the `dataset/` root directory, reads each subdirectory name, splits it on underscores, and verifies that every token belongs to the 12-class vocabulary defined in `CLASS_NAMES`. For each valid subdirectory a 12-dimensional multi-hot binary vector is constructed by setting

position k to 1 whenever class k appears in the folder name. Each image inside the subdirectory is then paired with this label vector, producing an entry in the `all_samples` list—a flat list of (`Path`, `torch.Tensor`) tuples that serves as the shared data structure for all downstream stages.

All images are resized to 128×128 pixels and normalized with the ImageNet channel-wise statistics: mean $\mu = (0.485, 0.456, 0.406)$ and standard deviation $\sigma = (0.229, 0.224, 0.225)$. These values are required because the EfficientNet-B0 backbone was pretrained on ImageNet and expects inputs in that standardized distribution. During training, two stochastic augmentation operations are applied to reduce overfitting: (i) random horizontal flipping with probability 0.5, and (ii) `ColorJitter` with brightness, contrast, and saturation perturbations drawn independently from $[0.8, 1.2]$ and a hue perturbation bounded within $[-0.05, 0.05]$. At validation and test time only the deterministic resize-and-normalize pipeline is applied, ensuring reproducible evaluation.

The complete training transform pipeline, applied in sequence, is as follows:

```
Resize(128x128)
RandomHorizontalFlip(p=0.5)
ColorJitter(brightness=(0.8,1.2), contrast=(0.8,1.2),
            saturation=(0.8,1.2), hue=(-0.05, 0.05))
ToTensor()
Normalize(mean=[0.485, 0.456, 0.406],
          std=[0.229, 0.224, 0.225])
```

The evaluation transform omits all stochastic steps, applying only `Resize`, `ToTensor`, and `Normalize`.

The 4,543-image dataset is partitioned using the ratio `prop_vec` $= [16, 2, 2]$, corresponding to an 80%/10%/10% split. Two successive calls to `sklearn.model_selection.train_test_split` with `random_state` $= 42$ yield 3,634 training images, 454 validation images, and 455 test images. The fixed random seed guarantees that every run of the notebook produces the identical partition, enabling reproducible comparisons across team members and experimental configurations.

Baseline Model

A logistic regression baseline is trained to establish a reference lower bound for the deep learning approach. Each image is resized to 32×32 pixels and its three RGB channels are flattened into a 3,072-dimensional feature vector of raw pixel intensities; no feature engineering or dimensionality reduction is applied. Scikit-learn's `OneVsRestClassifier` [3] is then used to fit one independent binary `LogisticRegression` per class, with regularisation strength $C = 0.1$, the `liblinear` solver, and a maximum of 200 optimization iterations. All twelve classifiers are trained in parallel via `n_jobs=-1`. At inference time, each binary classifier independently predicts whether its assigned class is present, and the twelve predictions are concatenated to form the full multi-label output.

This design is intentionally simple: it makes no assumptions about spatial structure, inter-class dependencies, or learned representations. Its sole purpose is to quantify the level of performance achievable with cheap pixel statistics, providing a concrete baseline against which the deep learning approach must demonstrate clear improvement.

Main Model: EfficientNet-B0 with Transfer Learning

The primary model is EfficientNet-B0 [4], a compound-scaled convolutional neural network that achieves a favourable accuracy-to-parameter-count trade-off on standard image benchmarks. We load the architecture with weights pretrained on ImageNet-1K (`EfficientNet_B0_Weights.IMAGENET1K_V1`) and replace the final linear layer of the classifier head with a new `nn.Linear(1280, 12)` layer, where 1,280 is the penultimate feature dimension of EfficientNet-B0 and 12 is the number of target classes. The modified head outputs 12 raw logits; at inference time a sigmoid function converts each logit to an independent class probability, and any class whose probability exceeds the threshold $\tau = 0.5$ is predicted as present.

Transfer learning is well-suited to this problem. Training a deep convolutional network from random initialisation on only 3,634 images risks both underfitting (insufficient capacity adaptation) and overfitting (lack of example diversity). Pretraining on ImageNet provides the backbone with rich general-purpose visual representations—low-level features such as edges and textures as well as higher-level object-part detectors—that transfer effectively to workspace object images even though the target label spaces differ. Fine-tuning all layers jointly then adapts these representations to the specific visual content of the dataset while retaining the broad knowledge acquired from ImageNet.

Training uses Binary Cross-Entropy with Logits Loss (`BCEWithLogitsLoss`), which combines sigmoid activation and binary cross-entropy in a numerically stable formulation and treats each of the 12 class outputs as an independent binary prediction. Optimization is performed with the Adam algorithm [1] at an initial learning rate of $\eta = 10^{-4}$. A step learning-rate schedule (`StepLR`) halves the learning rate every 5 epochs ($\gamma = 0.5$), enabling rapid convergence in early epochs followed by finer parameter refinement later in training. The model is trained for 20 epochs with a batch

size of 32, and the model's state dictionary is written to `best_model.pth` whenever the validation Macro F1 surpasses the best value recorded in any previous epoch. Training was conducted on the University of Florida's HiPerGator high-performance computing cluster, leveraging an NVIDIA CUDA GPU via SLURM job scheduling, which allowed all 20 epochs to complete in under 10 minutes.

During inference, the model is instantiated without pretrained weights and loaded entirely from the saved checkpoint, ensuring the evaluation pipeline runs completely offline with no external network dependencies.

3 Evaluation Methodology

Multi-Label Evaluation Metrics

Standard single-label accuracy is poorly suited to multi-label problems because it awards no credit for partial correctness: a classifier that correctly identifies 11 of 12 labels for every sample would score 0% under exact-match accuracy even though it is nearly right. We therefore adopt four complementary metrics that together characterize model quality from both strict and lenient perspectives.

Exact Match Accuracy (EMA) is the most demanding metric. A prediction is counted as correct only when all $K = 12$ label positions simultaneously match the ground truth:

$$\text{EMA} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\hat{\mathbf{y}}_i = \mathbf{y}_i] \,, \tag{1}$$

where N is the number of test samples, $\hat{\mathbf{y}}_i \in \{0,1\}^K$ is the predicted multi-hot vector, and $\mathbf{y}_i \in \{0,1\}^K$ is the ground-truth vector.

Hamming Loss measures prediction error at the individual label–sample level. It is defined as the fraction of all $N \times K$ label predictions that are incorrect:

$$L_H = \frac{1}{N \cdot K} \sum_{i=1}^{N} \sum_{k=1}^{K} \mathbf{1}[\hat{y}_{ik} \neq y_{ik}] \,. \tag{2}$$

A perfect classifier achieves $L_H = 0$; lower values indicate better performance. Unlike EMA, Hamming Loss gives partial credit when some but not all labels are correctly predicted.

Macro F1 computes the harmonic mean of precision and recall for each class independently and averages equally over all K classes. Because every class contributes the same weight regardless of frequency, Macro F1 is sensitive to poor performance on rare classes such as *clock* (the least frequent class), ensuring the model is not rewarded for ignoring underrepresented categories.

Micro F1 aggregates true positives (TP), false positives (FP), and false negatives (FN) across all class–sample pairs before computing F1:

$$F_{1,\text{micro}} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}} \,. \tag{3}$$

Because it pools statistics globally, Micro F1 is dominated by the most frequent classes and thus complements Macro F1, which treats all classes equally.

Data Splitting and Checkpoint Protocol

All hyperparameter decisions—including learning rate, batch size, scheduler parameters, number of training epochs, and classification threshold—are based exclusively on validation set performance. The test set is withheld and evaluated exactly once, after training is complete, to provide an unbiased estimate of generalisation performance. The checkpoint criterion is validation Macro F1: the model's state dictionary is written to `best_model.pth` whenever this metric improves. Final evaluation loads this checkpoint, applies the deterministic evaluation transform (resize + ImageNet normalization), runs inference across the test set, and computes all four metrics above. The graders additionally evaluate the submitted `best_model.pth` on an independent held-out test set via `eval.py`, using the same normalization statistics and a sigmoid threshold of 0.5. Per course requirements, the evaluation script must run completely offline; this is satisfied by instantiating the model with `pretrained=False` and loading it exclusively from the saved state dictionary, eliminating any network calls during inference.

Baseline Comparison and Tooling

The logistic regression baseline is evaluated on the same test partition as EfficientNet-B0, making the two comparisons directly valid. Any deep model that fails to outperform the baseline on all reported metrics would not justify its added

computational complexity and would require redesign. All metric computations use scikit-learn's `hamming_loss` and `f1_score` implementations [3], which adhere to the standard formulations in Equations (1)–(3). Loss computations during training use PyTorch's `BCEWithLogitsLoss` [2]. The baseline is evaluated on Hamming Loss and Macro F1; Exact Match Accuracy and Micro F1 are computed for EfficientNet-B0 only, keeping the baseline evaluation focused on the primary comparison metrics.

4 Results

Overall performance. EfficientNet-B0 substantially outperforms the logistic regression baseline across all jointly evaluated metrics, as shown in Table 1. The baseline achieves a Hamming Loss of 0.1474 and Macro F1 of only 0.1614 on the validation set—confirming that raw pixel features carry insufficient discriminative information for reliable multi-label prediction. Without a spatial hierarchy, learned convolutional filters, or any shared representation across classes, the one-versus-rest logistic regression approach learns only crude global intensity patterns, and its twelve independent classifiers cannot exploit the visual co-occurrence structure present in workspace scenes. By contrast, EfficientNet-B0 achieves a Macro F1 of 0.8127 on the held-out test set—a nearly five-fold improvement—along with a Hamming Loss of 0.0516 (2.9× lower than the baseline), Micro F1 of 0.8100, and Exact Match Accuracy of 0.5868. These results confirm that transfer learning from ImageNet provides the backbone with feature representations that generalise effectively to workspace object images and that the deep pipeline is well-calibrated for this multi-label task.

Table 1: Performance comparison of baseline and EfficientNet-B0. The baseline is evaluated on the validation set; EfficientNet-B0 is evaluated on the held-out test set. (↓ lower is better; ↑ higher is better.)

Model	Exact Match ↑	Hamming Loss ↓	Macro F1 ↑	Micro F1 ↑
Logistic Regression (baseline)	—	0.1474	0.1614	—
EfficientNet-B0 (ours)	0.5868	0.0516	0.8127	0.8100

Per-class analysis. Table 2 reports per-class F1 scores for EfficientNet-B0 on the test set, and Figure 1 visualises the distribution. Performance varies in a pattern that reflects both visual distinctiveness and class frequency. The strongest classes are *backpack* (F1 = 0.9322) and *chair* (F1 = 0.8870), both of which present distinctive silhouettes and large spatial footprints reliably discriminated across varied backgrounds. *calculator* (F1 = 0.8793) and *clock* (F1 = 0.8421) also perform well; notably, *clock* achieves this high score despite being the rarest class in the dataset (503 training images), suggesting that the pretrained features support strong generalisation even with limited training examples. The weakest classes are *phone* (F1 = 0.7286) and *bottle* (F1 = 0.7414). These lower scores are consistent with visual ambiguity: *phone* shares a flat rectangular appearance with *calculator* and the two frequently co-occur in workspace scenes, increasing false positives; *bottle* varies widely in shape, colour, and opacity and is often partially occluded by other desk objects, increasing false negatives.

Table 2: Per-class F1 scores for EfficientNet-B0 on the held-out test set.

Class	F1	Class	F1
pen	0.8333	chair	0.8870
paper	0.8085	desk	0.7742
book	0.7550	bottle	0.7414
clock	0.8421	keychain	0.7656
phone	0.7286	backpack	0.9322
laptop	0.8058	calculator	0.8793

Training dynamics. Figure 2 shows the evolution of training loss and validation Macro F1 over all 20 epochs. The model converges rapidly in the first five epochs: validation Macro F1 rises from 0.3443 at epoch 1 to 0.7576 at epoch 5, reflecting fast adaptation of pretrained ImageNet features to workspace-object representations. Following the first learning-rate decay at epoch 5 (from 10^{-4} to 5×10^{-5}), improvement continues more gradually as the optimizer makes finer weight adjustments. A second decay at epoch 10 (to 2.5×10^{-5}) yields further refinement, with the best checkpoint saved at epoch 17 (validation Macro F1 = 0.8067). After epoch 17 the metric fluctuates within ±0.004, indicating that the model has reached convergence. The training loss continues to decrease throughout all 20 epochs while the validation loss stabilises around 0.14, indicating mild overfitting that does not measurably harm test performance.

```
Loaded "best_model.pth"  (best val F1 during training: 0.8067)

=======================================================
Test Set Evaluation -- EfficientNet-B0
=======================================================
  Test Loss        : 0.1476
  Exact Match Acc  : 0.5868
  Hamming Loss     : 0.0516
  Macro F1         : 0.8127
  Micro F1         : 0.8100
-------------------------------------------------------
  Per-class F1:
    pen          0.8333
    paper        0.8085
    book         0.7550
    clock        0.8421
    phone        0.7286
    laptop       0.8058
    chair        0.8870
    desk         0.7742
    bottle       0.7414
    keychain     0.7656
    backpack     0.9322
    calculator   0.8793
=======================================================
```

Figure 1: Test set evaluation output showing per-class F1 scores for all 12 classes.

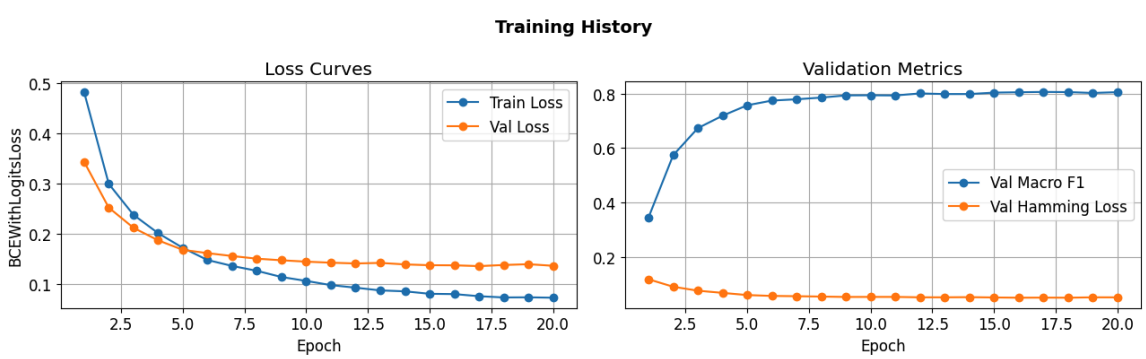

Figure 2: Training loss and validation metrics over 20 epochs on HiPerGator.

Dataset characteristics. Figure 3 shows a representative selection of images from the dataset. The images capture real-world workspace and desk environments with varying lighting conditions, camera angles, and background clutter, confirming that the model must learn robust object representations rather than exploit systematic imaging artifacts. With an average of 1.76 labels per image and a maximum of 7 simultaneous labels, the dataset exhibits the multi-label structure that motivated the use of per-class sigmoid thresholding rather than a single-class softmax output. Figure 4 shows the distribution of images across classes. Per-class counts range from 503 (*clock*) to 765 (*pen*), with the remaining ten classes distributed between 588 and 736 images. Despite this moderate imbalance, *clock* achieves one of the highest F1 scores (0.8421), indicating that the level of class imbalance here is not severe enough to necessitate explicit resampling or a weighted loss function.

Figure 3: Sample images from the dataset with their ground-truth labels.

```
Per-class image counts (sorted high to low):
========================================
  pen           765  |||||||||||||||||||
  bottle        736  |||||||||||||||||||
  laptop        728  |||||||||||||||||||
  book          728  |||||||||||||||||||
  paper         713  ||||||||||||||||||
  phone         711  ||||||||||||||||||
  calculator    648  ||||||||||||||||
  chair         627  ||||||||||||||||
  keychain      626  ||||||||||||||||
  backpack      614  ||||||||||||||||
  desk          588  ||||||||||||||
  clock         503  ||||||||||||

Avg labels per image: 1.76
Min labels per image: 1
Max labels per image: 7
```

Figure 4: Per-class image counts in the dataset (sorted by frequency). Clock is least represented (503); pen is most represented (765).

5 Conclusions

This project presents a complete end-to-end pipeline for multi-label image classification of twelve workspace object categories. EfficientNet-B0 fine-tuned on 3,634 training images achieves a test-set Macro F1 of 0.8127, Micro F1 of 0.8100, Hamming Loss of 0.0516, and Exact Match Accuracy of 0.5868—a dramatic improvement over the logistic regression baseline, which attains a Macro F1 of only 0.1614 and Hamming Loss of 0.1474. The pipeline satisfies all offline inference requirements: the evaluation script loads the model entirely from the saved `best_model.pth` state dictionary with `pretrained=False`, requiring no network access, and the grader-provided `eval.py` script can be run immediately after submission without any additional setup.

The central design choice driving these results is transfer learning from ImageNet [4]. Pretraining endows the EfficientNet-B0 backbone with rich visual feature representations—edges, textures, and object-part detectors—that transfer effectively to workspace objects, enabling fast convergence: validation Macro F1 reaches 0.76 within just five epochs on a training set of under 3,700 images. Data augmentation (random horizontal flips and `ColorJitter`) provided meaningful regularisation, preventing the mild overfitting observable in the training curves from significantly degrading test performance. The use of `BCEWithLogitsLoss` with per-class sigmoid thresholding embeds the correct inductive bias for this problem: since class memberships are independent, treating each output as a separate binary prediction is more principled than competing approaches such as multi-class softmax. The two-stage `StepLR` schedule also contributed meaningfully to convergence; both learning-rate decays produced measurable validation F1 improvements, confirming the value of staged step-size reduction after an initial fast-learning phase.

Of the reported metrics, Exact Match Accuracy (0.5868) is the weakest, which is expected: it requires every one of the 12 label positions to be simultaneously correct, and images with many active labels (up to seven in this dataset) offer more opportunities for a single misprediction. Several directions could improve results further. First, per-class threshold calibration—selecting a separate decision threshold τ_k for each class k based on validation-set precision–recall curves—could improve Macro F1, particularly for the weaker classes *phone* (F1 = 0.7286) and *bottle* (F1 = 0.7414), whose optimal thresholds likely differ from the global $\tau = 0.5$. Second, stronger augmentation strategies such as random cropping, MixUp, or CutMix could further reduce overfitting on the relatively small training set. Third, a larger backbone such as EfficientNet-B3 or a vision transformer architecture would increase model capacity at the cost of longer training time, potentially recovering the remaining gap between validation and test F1. Finally, the moderate class imbalance (503 images for *clock* vs. 765 for *pen*) was not explicitly addressed; a class-frequency-weighted loss or targeted oversampling of underrepresented label combinations could further improve performance on the rarest classes.

References

[1] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[2] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[3] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[4] Mingxing Tan and Quoc V Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 6105–6114. PMLR, 2019.